FairShare

Architecture and working MVP

Student teamwork: clear tasks, manageable workload, and visible obstacles.

Based on M1 Product Vision · October 2, 2026 · English edition October 3, 2026

01 Architecture

02 Team features

03 Allocation algorithm

04 Data and permissions

05 Roadmap and hypotheses

06 Sources

07 Visual specification

Includes source code, an API contract, and the SQL schema. Implemented capabilities are distinguished from future work; user research results are not fabricated.

FairShare architecture

Design date: October 2, 2026. Based on the completed M1 document, pages 2–4. This document describes the implemented MVP and identifies future work separately. VPS deployment was completed on October 3, 2026.

1. Product and constraints

FairShare helps students turn a course milestone into understandable tasks, agree on workload, and make obstacles visible early. The main workflow is **milestone → tasks with context → availability and skills → recommendation → discussion and approval → work → result**.

Observations in the product vision PDF come from the team's previous projects. Acceptance of the algorithm and the accuracy of hour estimates remain hypotheses. Neither hours nor activity counts are objective measures of contribution. Avoiding public inactivity rankings, automated grades, and accusations is a product decision, rather than a database limitation.

The initial working assumption is small student teams, several parallel projects, and a single weekly planning interval. Scale limits have not been validated through load testing.

2. Technology choices for this product

Option	Strengths	Limitations	Decision
React + Vite + separate API	Suitable for boards, forms, and shared workspaces; clear client/server boundaries	Sessions and API behavior need explicit implementation	Selected
Next.js	SSR, public pages, server rendering, and the React ecosystem	SSR offers limited product value for a private board and adds another server mechanism	A good alternative if a public portal is introduced
Vue / Svelte	Suitable for the same product	Switching would not improve the core workflow; the user preferred React	Not needed for this version
Flutter / React Native	Native mobile workflows and push notifications	Installation and separate platform testing increase the cost of the first experiment	Consider after validating mobile demand
Electron / Tauri	Desktop application support	No requirement for local files, an IDE, or system APIs	Not required
Firebase / Firestore	Quick authentication and synchronization setup	Team relationships, tasks, dependencies, versions, and atomic assignment fit a relational model better	PostgreSQL is a better fit for the data

React's official documentation recommends starting with a framework. This project deliberately uses a Vite SPA: the educational product has a separate API, a private workspace, no SEO requirement, and can validate its main workflow without SSR. This is a FairShare-specific decision, rather than a general claim about React.

Implemented technologies

Layer	Implementation	Rationale
Client	React, TypeScript, Vite, CSS tokens, Inter, Lucide	Interactive board, reusable components, and readable types
HTTP API	Node.js, Fastify, Zod	Small server, boundary validation, and explicit feature modules
Domain	Pure TypeScript allocation function	Testable independently of HTTP and the database
Data	PostgreSQL: local PGlite and pg for an external database	Shared SQL and transactional workflows without requiring Docker locally
Identity	Email/password, scrypt, random server sessions	Working local authentication without external accounts or mandatory API keys
Synchronization	REST and 15-second polling	Simple initial consistency model; user changes trigger immediate refetching
Verification	node:test, Fastify inject, Playwright Chromium	Domain, API, and real interface checks

Exact installed versions are recorded in package-lock.json. Any compatible provider may supply managed PostgreSQL. Supabase was investigated as a PostgreSQL/Auth option; the code does not use its SDK or require a Supabase project. If its database is chosen, the server connects through pg; the Data API must not expose these tables. Supabase Auth would be a future replacement for the Identity module, not an implemented feature.

3. Architectural style

A **modular monolith**: one server process, one relational database, and several functional modules. This suits a team of two or three developers. Distributed transactions, separate queues, and microservice contract coordination would add little value at this stage.

The implementation uses separation of concerns, typed boundaries, UI composition, an independent domain algorithm, a shared Sql/Database port, transactional workflows, optimistic locking for tasks, and explicit authorization. Calling this a strict Clean Architecture implementation would be inaccurate: SQL and application workflows sit alongside HTTP handlers inside feature modules. As the system grows, they can move into application/ and repositories/ while preserving the HTTP contract and domain function.

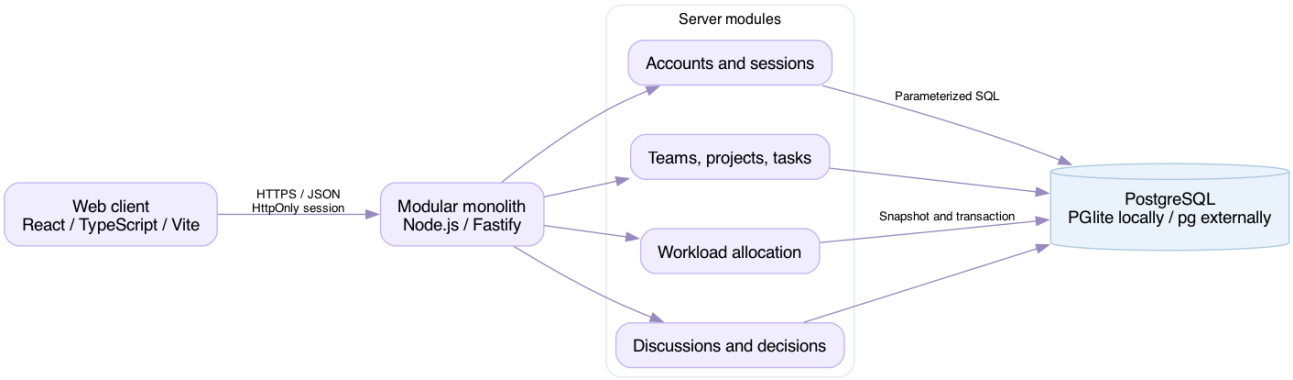
System context

Mermaid diagram; source is available in the corresponding Markdown document.

External documents remain with their owners; FairShare stores only a link and title. The application does not download code or assess its quality.

Containers and data

Mermaid diagram; source is available in the corresponding Markdown document.



These are C4-style Mermaid diagrams, rather than formal UML. A static SVG container diagram supports reading outside Markdown. The surrounding text provides an accessible alternative. Wide SVGs should scroll instead of shrinking until their labels become unreadable.

4. Modules and ownership

Module	Responsibility	Main files
Identity	Accounts, passwords, sessions, demo login	server/auth.ts, server/security.ts
Workspace	Project access, member lists, aggregated workload	server/workspace.ts
Projects	Teams, projects, milestones, invitations, availability	server/projects.ts
Tasks	Task context, statuses, dependencies, help, results	server/tasks.ts
Allocation	Proposals and transactional approval	server/allocation.ts, server/allocations.ts
Collaboration	Discussions, steps, resources, decisions, votes	server/collaboration.ts
Persistence	Shared SQL port, adapters, schema, educational seed	server/db.ts, server/schema.sql, server/seed.ts
Web	Navigation, workspace views, dialogs	src/components, src/features
Contracts	Shared client/server types and OpenAPI	shared/types.ts, contracts/openapi.yaml

In a larger version, application workflows should depend on repository ports rather than importing Fastify. The allocation domain already respects this boundary: it knows nothing about cookies, HTTP, PostgreSQL, or React.

5. Allocation approval workflow

Mermaid diagram; source is available in the corresponding Markdown document.

Membership, availability, and existing assignments are shared across projects. Team and user records are locked before changes; user locks follow a stable UUID order to reduce deadlock risk. Availability updates lock the user's record. Approval rereads the snapshot under those locks. Protection is enforced by the server regardless of the client button's state.

The snapshot includes milestone revision, task IDs and versions, team membership, skills, available hours, and workload. A change to another project's task in the same week changes workload, invalidating an earlier proposal. Approving an already approved proposal returns success without assigning tasks again: the operation is idempotent.

6. Client architecture

App coordinates sessions and project/milestone selection; AppShell handles navigation. The board, workload, team, personal tasks, and activity log are separate feature components. Shared Modal, Button, Avatar, and Progress components provide consistent behavior.

The API client uses same-origin cookies. Saved data is reread from the server, avoiding disagreement between an optimistic interface and server authorization. A request counter protects the main workspace against stale responses when switching projects. A larger version should use TanStack Query with keys such as [projectId, milestoneId], request cancellation, deduplication, and centralized invalidation.

Drag-and-drop is not required: task status changes through an accessible form. Keyboard controls, dialog focus handling, Escape dismissal, mobile layout, text contrast, and reduced motion are supported. These are implemented measures; a separate audit is required before claiming full WCAG certification.

7. Hosting and growth

Locally, one Node process serves the React build and API; PGlite stores data on disk. Do not open a PGlite directory from several processes or place it on ephemeral hosting storage.

The deployed VPS runs the static build and Node API behind Nginx HTTPS, with separate PostgreSQL, a dedicated application account, systemd startup/recovery, daily backups, and a tested release rollback. See VPS operations.

For a pilot, maintain separate demo/staging/production environments and manage environment secrets. Keeping React and the API on one origin reduces cookie and CORS complexity. Hosting providers remain independent of the product domain.

Horizontal scaling first requires regular PostgreSQL, shared rate limiting, and consistent session rules. Push synchronization can use SSE/WebSocket with membership checks and a durable event cursor. If email or notifications are added, save the event and an outbox entry in one transaction; a worker delivers them with retries. Redis becomes useful when queue or presence demand is established. Kubernetes is not a launch requirement.

8. Non-functional targets and current boundaries

Pilot targets, not validated load measurements: ordinary API p95 below 500 ms with 50 active users; allocation of 100 tasks across 10 members within one second; synchronization of other users' changes within 15 seconds plus network latency; production database recovery within four hours, with up to 24

hours of data loss under daily backups.

Implemented capabilities include transactional approval, version checks, membership authorization, sessions, activity history, VPS hosting, and tested database backup restoration. Password recovery, email verification, file storage, email/push delivery, comprehensive versioned migrations for an existing production database, individual member approval of assignments, and multi-instance infrastructure remain future work. They are described in the roadmap.

Features for team collaboration

“Implemented” means available in both the API and interface. Other items describe future development, rather than existing code. This catalog is based on M1 and the additional request for broader team functionality.

Implemented in the MVP

Feature	Benefit for the team
Registration, login, logout, and persistent sessions	Personal permissions and commitments
A user's own team and membership in multiple teams	Different study groups in one application
Single-use invitations valid for 24 hours	Members can join independently
Multiple projects and weekly milestones	Separate courses and intermediate deliverables
Task context: goal, first step, deliverable	A clear starting point without repeated coordinator explanations
Estimated hours and required skill	A discussable scope and feasible work
Weekly personal availability	Accounts for classes and other commitments
Self-declared skills	Transparent assignment constraints
Shared workload across all projects	The same free hours are not spent several times
Explainable allocation proposals	A starting point for discussion
Coordinator approval after discussion	The team retains decision-making authority
Protection against stale proposals	New tasks and availability changes are considered
Manual assignment with exception confirmation	Flexibility with visible overcommitment risk
A three-state board	A shared view of progress
Personal tasks across all projects	Clear individual commitments
Task dependencies and cycle protection	Explicit work order
One-click help requests with explanations	Early visibility of obstacles
Result links or descriptions	Recorded work outcomes
Task-specific discussions	Clarifications stay with the task instead of being lost in chat
Shared checklists of concrete steps	Easier task starts and handoffs
Shared document and repository links	Resources accessible from the project

Feature	Benefit for the team
Team decision proposals	Questions are stated explicitly
Yes/no voting and vote changes	One current vote per participant
Coordinator-recorded decision outcomes	Agreements remain available
Recent activity log	Change context without ranking people
Refresh every 15 seconds and on window focus	Visibility of other members' changes
Responsive interface and keyboard-accessible dialogs	Mobile use without requiring drag-and-drop

P1: day-to-day coordination

Capability	Scenario	Architectural addition
Individual assignment acceptance	“Accept” / “Request changes”	assignment_responses and a plan activation rule
Task handoff with context	A member requests a replacement and shares completed work	handoff_requests and recipient consent
Pair work	A newcomer works with a mentor	task_collaborators, with one accountable owner
Peer review	An author submits a result to a reviewer	review_requests and an in_review state
Acceptance criteria	The team agrees on completion in advance	Required acceptance checklist items
Estimate ranges and confidence	Account for underestimation risk	estimate_low/high and workload scenarios
Collaborative estimation	Members estimate independently	estimate_votes, median, and spread
Hour reserves	Allow for mistakes and learning	capacity_reserve
Multiple dependencies and a graph	Make delivery chains visible	task_dependencies and DAG validation
Milestone templates	Repeated labs and presentations	Versioned templates
Starting-context templates	Examples for UI, API, tests, and reports	First steps, files, and sample deliverables
Search and filters	Filter by person, status, skill, or due date	Pagination, server filters, and full-text search
Semester archive	Completed work does not distract from current work	archived_at and a history policy

Capability	Scenario	Architectural addition
Project and milestone editing	Refine names and dates	Version checks and workload recalculation
Mentions	Direct requests for help	Membership checks and rate limits
Task subscriptions	Follow an important dependency	task_subscriptions
Notification center	Assignments, replies, and help requests	Notifications, a read cursor, and deduplication
Personal reminders	Members choose their own timing	Notification preferences, outbox, and worker
Team digest	One summary instead of many messages	Scheduler and channel selection

P2: agreements and learning

Capability	Benefit	Conditions
Meeting agenda	Questions are ready before discussion	Links to milestones and tasks
Meeting outcomes	Conversations become actions	Decisions kept separate from new tasks
Meeting-time poll	Choose a convenient slot	Time zones and multiple options
Asynchronous check-in	“Done / next / blocked”	Brief and voluntary, without timesheets
Risk board	Problems surface before deadlines	Impact, likelihood, and a response owner
Critical path	Delaying chains are visible	Actual dependencies and durations
Retrospective	Preserve useful experience	One agreed experiment for the next milestone
Knowledge base	Examples and decisions stay with the team	Search and ownership of updates
Team agreement	Shared expectations about replies and help	Versions and member acceptance
Rotation of learning roles	Share coordination and development	Learning roles separate from system permissions
Personal learning goals	Feasible tasks for a new skill	Distinguish “want to learn” from “can do”
Private availability explanations	Members do not have to disclose reasons to everyone	Total available hours are sufficient for planning

Capability	Benefit	Conditions
Handoff and re-estimation history	Agreements can be reconstructed	Author, reason, and approval
Project export	Share the team's result	Authorized data only
Read-only mentor	Show progress to an instructor	Limited access and revocable invitations

P3: integrations

- GitHub/GitLab: attach results and PR checks; commit counts do not measure quality or contribution.

- Files: private object storage, limits, scanning, and signed URLs.

- Shared deadline calendar, ICS, and calendar synchronization; synchronization is excluded from the first M1 version.

- Telegram/email digests: only after a member explicitly subscribes to a channel.

- PWA, offline reading, and later conflict-safe offline changes.

- AI-generated task breakdown drafts with human review; AI syllabus parsing is excluded from the first M1 version.

- Anonymized product metrics for hypothesis validation, without public student rankings.

- CP-SAT/integer optimization if real data shows that the greedy heuristic is insufficient.

Product boundary

A full messenger, IDE, automatic code grading, mandatory daily timesheets, public inactivity rankings, and academic grading are outside FairShare's scope. Contextual discussions support task completion; the team's main chat can remain in its existing tool.

Recommended order after the MVP: individual acceptance → handoffs → peer review → notifications → templates. Add complex integrations when demand is observed, so the application does not increase administrative work.

Explainable workload allocation

Inputs and the definition of fairness

Planning covers one week starting on Monday UTC. capacity(member, week) is the number of hours the member declares for **all** projects in that week, rather than a separate budget for each project. Missing availability is treated as zero: the application does not invent free time.

load(member, week) is the sum of estimates for the member's assigned, unfinished tasks in milestones for that week. Completed task counts, page views, and comments are not used as contribution scores. A blocked task continues to consume planned hours until the team moves or changes it.

The workload ratio is r = planned_hours / declared_hours. For example, six out of six hours means full capacity, while six out of twelve means half capacity. Equal absolute hours or card counts would produce a different, less appropriate allocation.

Implemented MVP algorithm

1. Consider only planned tasks in the selected milestone. Work in progress keeps its assignee; completed tasks are excluded.

2. Subtract the existing assignments of those planned tasks from total workload, so recalculating a milestone does not count them twice.

3. Find eligible members for each task: positive availability and the required skill. The “Any skill” category skips the skill check.

4. Sort tasks by the fewest eligible members, then the largest estimate, then UUID. This protects scarce-skill tasks from earlier consumption of the capacity they need.

5. Exclude members who would exceed availability if assigned the entire task. The algorithm cannot partially assign an indivisible task.

6. Choose the smallest projected ratio (existing_load + task_estimate) / capacity. Break ties using a stable UUID order.

7. If no member is eligible, leave the task unassigned and explain whether the problem is a missing skill or insufficient hours.

8. Show projected workload and assignment reasons. Save the proposal and input snapshot. Actual assignments remain unchanged until approval.

9. The coordinator approves the recommendation after team discussion. This is not individual digital consent from every member; that mechanism is planned for a later version.

The function is in server/allocation.ts and has no database or HTTP dependency. The current straightforward implementation has approximately O(T × M log M + T log T) complexity plus linear passes, which is adequate for small teams.

Example

Yerassyl has 12 hours and Backend/Architecture skills; Suusar has eight hours and Frontend/Testing; Sezim has six hours and Documentation/Databases. Existing work includes five UI hours for Suusar and

two presentation hours for Sezim. A four-hour architecture task and six-hour API task fit Yerassyl; a three-hour testing task fits Suusar.

Member	Existing work	Proposed tasks	Total	Availability
Yerassyl	0	4 + 6	10	12
Suusar	5	3	8	8
Sezim	2	0	2	6

Hours are unequal because skills restrict eligible assignments. The application should not artificially assign Sezim a technical task she has not declared herself able to perform.

Exceptions and uncertainty

- Pairing, learning support, or splitting tasks could help when skills are missing. These are useful future features. The MVP exposes the issue and leaves the decision to the team.

- Existing overcommitment remains visible; the algorithm does not assign additional tasks beyond available hours.

- Manual assignment can override restrictions only after explicit coordinator confirmation; the exception is recorded in the activity log.

- Hour estimates are unreliable. A later version should add ranges, confidence, team review, and reserves. Declared estimates must not be treated automatically as actual working time.

- Dependencies determine start/completion order, not a precise schedule within the week. The algorithm does not verify critical paths or calendar feasibility.

- A greedy heuristic can leave a task unassigned even when another order would yield a feasible plan. Larger teams could compare it with integer programming or CP-SAT; an optimizer is not implemented.

Consistency

All assignment changes are approved in a single transaction. External PostgreSQL uses team and member row locks; tasks also carry a version in the UI. A proposal signature includes tasks, workloads, hours, and skills. A mismatch returns HTTP 409. Reapproving an approved proposal does not reassign anything.

Tests cover zero availability, missing skills, other-project workload, preservation of work in progress, prevention of double counting, scarce skills, large indivisible tasks, different hours-to-availability ratios, and deterministic results.

Data, permissions, and operations

Entities

Table	Purpose	Constraints
users	Accounts and skills	Unique email, scrypt password hash
sessions	Server-side sessions	Token hash only, expiry, user FK
teams / memberships	Teams and roles	Team/user primary key, one coordinator per team
availability	Weekly hours	User/week primary key, 0–100 hours
projects / milestones	Projects and weekly milestones	Team and project FKs, milestone revision
tasks	Context and work progress	Composite milestone/project and dependency/project FKs, positive hours, version
allocations	Allocation proposals	Snapshot, creator, payload, approval state
invitations	Single-use invitations	Token hash, expiry, used_by
events	Activity history	Project, user, type, minimal context
comments / checklist	Discussions and concrete steps	Task FK, cascade deletion with the task
resources	External resources	Title, HTTP/HTTPS URL, author
decisions / votes	Team agreements	One vote per user/decision, explicit closure

Mermaid diagram; source is available in the corresponding Markdown document.

Executable schema: server/schema.sql. The API additionally checks calendar dates, Monday week starts, due dates within the selected week, 0.25-hour estimate increments, dependency cycles, assignee membership, and status conditions. Direct administrative database edits bypass rules that are not duplicated in SQL.

Task states

planned → in_progress → done. A task can return to planning or be completed directly if it has an assignee, its dependency is complete, and a result is described. Completion requires at least 10 characters of evidence text/link and no outstanding help request. Reopening a completed task requires availability checks. Help is a separate field and does not obscure the main status.

started_at records the declared start, first_evidence_at the first result description/link, and done_at completion. These are technical events; they do not automatically prove meaningful work.

Permissions

Action	Coordinator	Member
Read own team	Yes	Yes
Read another team	No	No
Create a task	Yes	Yes
Change task requirements and assignment	Yes	No
Change status, help, or result	Any task in the team	Own assigned tasks only
Delete a current planned task with no dependents	Yes	No
Change own skills/hours	Yes	Yes
Change another user's hours	No	No
Create a project, milestone, or invitation	Yes	No
Propose and approve allocation	Yes	No
Discuss and add steps/resources	Yes	Yes
Propose a decision or vote	Yes	Yes
Record a decision outcome	Yes	No

The server checks membership on every request. Knowing a UUID does not grant access. An object outside the user's team returns 404; a prohibited action returns 403. There is no superuser role in the user-facing API.

Implemented protections

- Passwords use individual salts and scrypt. Plaintext passwords are not stored in the database or logs.

- New passwords require 15–128 characters, permit spaces and Unicode, and have no mandatory character categories. Passwords are not trimmed. Existing shorter passwords remain valid on login. Email addresses are trimmed and lowercased; authentication validation is shared between client and server.

- Authenticated users can change their own password with the current password and matching confirmation. The server rejects reuse of the current password, revokes all existing sessions, and creates a fresh current-browser session atomically. User-row locking serializes password changes with session creation; the session is rechecked inside the change transaction. Shared demo passwords cannot be changed while demo mode is enabled. This is distinct from forgotten-password recovery, which still requires email delivery.

- Sessions use 32 random bytes, stored as SHA-256 hashes, and expire after seven days. Cookies are HttpOnly/SameSite=Strict, with Secure enabled in production. Successful authentication rotates the current browser session and revokes its previous token in the same transaction; failed login preserves the current session. Logout deletes the server session.

- Mutating requests check Origin and Sec-Fetch-Site; production requires the exact APP_ORIGIN. Arbitrary CORS is not enabled.

- Zod validation, parameterized SQL, and a 100 KB body limit. Login, registration, password changes, and enabled demo login share a limit of 12 requests per minute per IP within the current process, with an error message and Retry-After header. Unknown emails perform dummy scrypt verification; unknown-account and incorrect-password login failures return the same error.

- Only the loopback reverse proxy is trusted to supply client IPs. The deployed application listens on localhost behind Nginx.

- React escapes text. Shared links accept HTTP/HTTPS and open with noopener/noreferrer.

- API responses use no-store. Static responses include CSP, nosniff, no-referrer, and iframe DENY protections.

- Invitations are single-use, valid for 24 hours, and stored as hashes. Sharing an invitation remains the user's action.

- Registration can consume an invitation in the account-creation transaction. It grants member access to the invited team without creating an unrelated team. An invalid, expired, or used invitation rolls back account creation; normal login still accepts a pending invitation after authentication.

- Tasks carry versions; allocation approval is atomic and validates the snapshot signature. Team and member records are locked in a stable order.

- NODE_ENV=production disables demo login by default. Removing the route does not delete educational accounts: an external pilot needs a separate clean database.

Before a broader external pilot

Email verification, password recovery, retention/deletion policies, comprehensive error monitoring, versioned migrations, and separate runtime/migration credentials remain future work. The current schema creates tables idempotently; it is not a complete production schema evolution mechanism.

VPS backups and restoration were implemented and verified on October 3, 2026. Daily backups include the PostgreSQL dump, source, configuration, and checksums. A full restore was tested in a separate database. See the VPS guide for retention and download procedures.

If Supabase is selected as the database provider, disable its Data API for server-owned tables and client-role grants, or use a non-exposed schema. Exposed tables require RLS. Service/secret keys must never be included in React. This application protects data through API checks; the client does not access the database directly. Supabase Auth is not integrated.

Multiple API processes require regular PostgreSQL, shared rate limiting, and concurrency lock verification. PGlite is intended for a single process; do not open the same directory from two servers or store it on ephemeral hosting storage.

Retention and operations

The MVP does not provide account/project deletion, archiving, or discussion deletion. Session expiry is checked on each request, and expired sessions are removed at startup. Retention controls are part of the roadmap.

/api/health checks database connectivity. The recommended verification pipeline is npm ci → typecheck → test → build; it is not an installed GitHub Actions workflow. Back up local PGlite after a clean process shutdown; moving to regular PostgreSQL requires export/import.

The deployed VPS uses local PostgreSQL and a daily systemd backup timer. Broader hosting can use provider backups plus regular restore tests in a separate environment. Proposed RPO of 24 hours and RTO of four hours are pilot targets, not a measured SLA. VPS measures include key-only SSH, disabled root login, a firewall, fail2ban, automatic security updates without automatic reboots, swap, and log storage limits.

Roadmap and product validation

Already implemented

A working MVP: identity and teams, projects and weekly milestones, task context, availability/skills, recommendations and atomic approval, task board/help/results, discussions/checklists, resources, and team decisions. Code is organized into modules; PostgreSQL schema, adapters, types, and checks are in the repository.

On October 3, 2026, the MVP was deployed to a VPS with HTTPS, separate PostgreSQL, automatic startup/recovery, tested release rollback, and daily backups with verified restoration. The initial backup was also downloaded to the local computer. Operational commands are documented in the VPS guide.

Next stages

Stage	Outcome	Completion criterion
Pilot 1, one week	Two teams complete the core workflow	Measure first-step clarity, work starts, and setup time
Pilot 2, one to two weeks	Validate recommendation usefulness	Record acceptance, reasons for manual changes, and estimate errors
Agreement, one to two weeks	Individual assignment acceptance and handoffs	Members explicitly accept changes to their commitments
Peer review, one to two weeks	Authors submit results to reviewers	Keep feedback, corrections, and acceptance with the task
Operations, one to two weeks	Readiness for a limited external pilot	Verify email verification/reset, migrations, monitoring, and continued backup restoration
Usability, one to two weeks	Templates, search, and notifications	Reduce manual work without mandatory daily forms

Timelines are preliminary estimates for a team of two or three people, not commitments. Development and research can overlap. Calendar and AI integrations remain later work, as specified in M1.

Validating the M1 hypotheses

H1: clear context helps people start work

Participants: two real student teams, four to six students, with voluntary participation. The baseline is a comparable task in their usual chat; the intervention is a three-day milestone with six structured tasks. Members read the goal, first step, and expected deliverable before starting.

“Starting meaningful work” means the first substantive document, draft, or code commit within 48 hours, verified by a researcher. Moving a card to “In progress” alone is insufficient evidence. first_evidence_at helps locate when evidence was attached, while its substance and actual creation time are verified separately.

Measurements: time to the first artifact, the number of “where do I start?” questions, coordinator intervention, and members' ability to describe the first step and deliverable in their own words. The PDF's support criteria are that members start within 48 hours, ask noticeably fewer questions, and understand expectations. Additional thresholds can be agreed in advance, such as a 30% relative decrease in questions; this has not been tested and is not a factual result.

If members remain inactive, investigate skills, schedules, dependencies, and motivation separately. The application cannot compel effort.

H2: recommendations are accepted as a starting agreement

Record how many proposals are shown and approved, manual changes, and explanations for them. Ask which constraints the algorithm considered or missed. A coordinator's click does not prove everyone else's agreement; a brief team interview is needed.

H3: hour estimates are sufficient for planning

Compare the original estimate with a voluntary final effort estimate and qualitative reasons for discrepancies. Do not introduce mandatory daily timesheets. If underestimation is systematic, add ranges, reserves, and peer review of estimates. done_at - started_at measures elapsed calendar time, not actual hours worked.

Reducing administrative work

Measure the time to connect the first member and create the first understandable task. M1 targets initial onboarding below three minutes. Measure the entire project setup path separately: the three-minute target is not a promise for complete system setup.

Product metrics

Metric	Definition	What it does not prove
Starting-point clarity	Share of members who correctly explain the first step and deliverable	Motivation or completion
Start within 48 hours	Share of tasks with a manually verified first artifact	Final result quality
Recommendation acceptance	Share of approved proposals and reasons for changes	Whole-team agreement without interviews
Timely help	Time between a known obstacle and a help request	Absence of unrecorded obstacles
Planned overcommitment	Assignment hours beyond declared availability	Actual overload outside the application
User friction	Setup time and repeated actions	Market demand

Do not fabricate feedback, study results, or metric values. The project currently has educational data and technical verification; a user study has not been conducted.

Suggested team responsibilities

One member owns UI and accessibility; another owns API/data/domain; a third owns testing, research, documentation, and acceptance. For two people, split the third role. Ownership must not prevent shared review: everyone should be able to explain the product, workload model, and main checks.

Sources and design decisions

Sources were checked on October 2, 2026. research/sources.json records URLs, HTTP statuses, and retrieval times. The research script can retrieve text into research/*.txt; those text snapshots are not included in this repository. These checks concern official documentation, rather than user research.

Source	What was checked	Impact on FairShare
React: Creating a React App	React recommends frameworks and permits custom setups for appropriate constraints	A Vite SPA was deliberately selected for a private board with a separate API; Next.js remains an alternative
Vite: Guide	TypeScript/React templates and Node requirements	Reproducible builds and a compatible Node runtime
PostgreSQL: Explicit Locking	Row-level locks, FOR UPDATE, and deadlocks	Team and user locks follow a stable order
PostgreSQL: Transaction Isolation	Consistency across concurrent transactions	Snapshot rereading and atomic approval
OWASP: Authorization Cheat Sheet	Deny by default and check permissions on every request	The server checks team membership instead of relying on the UI
TanStack Query: Overview	Server state, caching, and synchronization	Recommended as the application grows; not integrated in the MVP
WCAG 2.2	Keyboard use, focus, accessibility, and responsive behavior	Focus traps, visible focus, and mobile layout, without a certification claim
Supabase: Securing your API	Schema exposure, grants, and the Data API	If selected as the PostgreSQL provider, server tables must not be exposed to browsers
Supabase: RLS	Row-level access policies	Exposed tables require RLS; a separate API does not enable it automatically
Supabase: JWTs	Token verification and signing keys	A future Identity replacement needs actual JWT verification; the MVP uses server-side sessions
Supabase: Changelog	Current and breaking changes	Cloud APIs are not assumed to remain unchanged

The recorded Supabase index reported PostgreSQL minor-version changes dated September 25, 2026, including warnings about reindexing some extensions and legacy-cipher data. The MVP does not use ltree, btree_gist, pgcrypto encryption, or an existing Supabase database; no migration of another project was performed. Signing-key documentation was also checked through official Supabase MCP search.

The NestJS page was retrieved but contained only a short JavaScript placeholder. It was not used as evidence for an architectural choice. Fastify was chosen for the size of the feature modules and simplicity of a small monolith; this is an engineering decision, not the result of a comparative benchmark.

Product boundaries and hypotheses come from the user-supplied PDF, rather than internet sources. The Gemini link in that PDF was not verified; its conclusions were treated as recorded team decisions, not as a new conversation with Gemini.

FairShare visual specification

Reference: diagrams/ui-concept.png, 1536 × 1024. This is an application workspace with an open board, three columns, and a separate right-hand workload area. The reference image shows the original Russian interface. There is no marketing page inside the workspace.

Tokens and geometry

- Main background: cool off-white #f7f8fb; surfaces and sidebar: #ffffff.

- Primary text: #171928; secondary text: #737d96; borders: #e5e8f0.

- Accent: #7056e8; light accent background: #eeebff; help background: #fff5df, text #946018.

- Font: Inter when available, otherwise system sans-serif; distinct sizes for headings, buttons, captions, and fields.

- Milestone heading: 38–40 px, weight 700; interface text: 14–16 px; captions: 12–14 px.

- Navigation: 278 px on a wide screen, 230 px on a laptop; top bar: 64 px; workspace padding: 30 px.

- Card radius: 12 px; button radius: 9 px. Subtle shadows, without gradients or decorative illustrations.

- Lucide icons: 18–22 px, stroke width 1.7–2. The equals sign is a meaningful UI icon; branding remains text.

Approved first-screen content

Labels on the first screen (the reference image shows an earlier Russian version):

FairShare; Work in balance; My projects; My tasks; How it works; PROJECTS; Software Engineering; Databases; Invite; ISE4133 · Team FairShare; First working prototype; Clear tasks. Manageable workload. Shared results.; Milestone due date; dynamic date; task, hour, and member counts; Start with a fair allocation; Compare workload with each person's available time.; Allocate tasks; Board; Workload; Team; New task; Planned; In progress; Done; Team workload; This week's plan; Ask for help in time; We count planned hours, not contribution or grades.; Open a task to see its first step and expected deliverable.

Task data and member names come from the API. Counts, dates, and workload change with the data: the design reference is not a source of business data.

Additional required states

- Login and registration, with demo mode labeled separately.

- Allocation dialog with explanations and explicit approval.

- Task dialog: goal, first step, deliverable, hour estimate, skill, assignee, dependency, state, help request, and result link/description.

- Project and milestone creation, plus milestone selection when several exist.

- Team view: editing personal availability and skills, and creating single-use invitations.

- Workload view: hour totals and ratios to availability.

- Loading, empty projects, no eligible assignees, stale proposals, and access errors.

These states are necessary functional extensions of the reference. On mobile, navigation moves behind a menu button, the board becomes vertical, and workload moves below it. Dialogs support keyboards: they trap focus, close with Escape, and return focus to their trigger. Visible focus rings are required. Reduced animation respects prefers-reduced-motion.